

Java Programming

UNIT - 1

Question 1: Explain various features of Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects," which contain data (attributes/fields) and code (methods). The primary goal of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

Core Features (The 4 Pillars):

1. Encapsulation:

- **Definition:** The wrapping up of data (variables) and methods (functions) into a single unit (class).
- **Purpose:** It achieves data hiding. Variables are typically declared as private and accessed via public getter and setter methods.

2. Abstraction:

- **Definition:** Hiding the complex internal implementation details and showing only the essential features to the user.
- **Implementation:** Achieved in Java using abstract classes and interfaces.

3. Inheritance:

- **Definition:** The mechanism by which one class (child/subclass) acquires the properties and behaviors of another class (parent/superclass).
- **Purpose:** Promotes code reusability and establishes a parent-child relationship (IS-A relationship) using the extends keyword.

4. Polymorphism:

- **Definition:** The ability of an object to take on many forms. It allows methods to perform different tasks based on the object that invokes them.
- **Types:** Compile-time polymorphism (Method Overloading) and Run-time polymorphism (Method Overriding).

Other Important Features:

- **Class:** A user-defined blueprint or prototype from which objects are created.
- **Object:** A real-world entity and an instance of a class that consumes memory when created.

Java Programming

Question 2: Explain static method, static field and static block with example.

Introduction to static keyword: In Java, the static keyword is primarily used for memory management. It indicates that the particular member belongs to the class itself, rather than to any specific instance (object) of that class.

1. Static Field (Static Variable):

- **Concept:** When a variable is declared as static, a single copy of that variable is created and shared among all objects at the class level.
- **Use Case:** Used for common properties of all objects (e.g., a company name for all employees or a college name for all students). It saves memory as it is allocated only once in the class area at the time of class loading.

2. Static Method:

- **Concept:** A method declared with the static keyword belongs to the class rather than the object.
- **Rules:** * It can be invoked without creating an instance of the class (e.g., `ClassName.methodName()`).
 - It can only directly access other static data and call other static methods. It cannot use `this` or `super` keywords.

3. Static Block:

- **Concept:** A block of code enclosed in `{}` preceded by the static keyword.
- **Execution:** It is used to initialize static data members. It is executed strictly once, automatically, when the class is loaded into memory, even *before* the main method executes.

Java Programming

Question 3: Discuss Java Operators with examples.

Definition: Operators in Java are special symbols used to perform specific operations (mathematical, logical, or relational) on one, two, or three operands, and then return a result.

Categories of Operators:

1. **Arithmetic Operators:** Used for basic mathematical operations. (+, -, *, /, %)
2. **Relational (Comparison) Operators:** Used to check the relationship between two operands. Returns a boolean value. (==, !=, >, <, >=, <=)
3. **Logical Operators:** Used to combine multiple boolean expressions. (&& Logical AND, || Logical OR, ! Logical NOT)
4. **Assignment Operators:** Used to assign values to variables. (=, +=, -=, *=)
5. **Unary Operators:** Require only one operand (e.g., ++ increment, -- decrement).
6. **Ternary Operator:** A shorthand for if-else statements. (condition ? true_value : false_value)

```
public class OperatorDemo {  
    Run | Debug  
    public static void main(String[] args) {  
        int a = 10, b = 5;  
  
        // Arithmetic  
        System.out.println("Addition: " + (a + b)); // 15  
  
        // Relational  
        System.out.println("Is a greater than b? " + (a > b)); // true  
  
        // Logical  
        System.out.println("Logical AND: " + (a > 0 && b > 0)); // true  
  
        // Ternary  
        int max = (a > b) ? a : b;  
        System.out.println("Max value is: " + max); // 10  
    }  
}
```

Java Programming

Question 4: Explain primitive and non-primitive data types in Java.

Data types in Java specify the size and type of values that can be stored in a variable. Java is a statically typed language, meaning all variables must be declared before they can be used. Data types are divided into two main categories: Primitive and Non-Primitive.

1. Primitive Data Types: These are the most basic data types built into the language. They hold their values directly in the memory where the variable is allocated (usually the Stack). Java has 8 primitive data types:

- **Integer types:** byte (1 byte), short (2 bytes), int (4 bytes), long (8 bytes).
- **Floating-point types:** float (4 bytes), double (8 bytes).
- **Character type:** char (2 bytes, uses Unicode).
- **Boolean type:** boolean (1 bit, true/false).

2. Non-Primitive (Reference) Data Types: These are created by the programmer (except for String, which is built-in but still an object). Instead of storing the actual value, they store a *reference* (memory address) to the object located in the Heap memory.

- **Examples:** Classes, Interfaces, Arrays, Strings.

Feature	Primitive Data Types	Non-Primitive Data Types
Creation	Predefined by Java.	Created by the programmer (except String).
Storage	Stores the actual value directly.	Stores a memory reference to an object.
Memory	Stored in Stack memory.	Stored in Heap memory.
Null Values	Cannot be null (have default values like 0 or false).	Can be assigned a null value.
Methods	Do not have methods.	Can have methods to perform operations.

Java Programming

UNIT – 2

Question 5: Define Object and Class with example.

1. Class:

- **Definition:** A class is a user-defined blueprint, template, or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type.
- **Nature:** It is a logical entity. A class does not consume any memory space when it is created.

2. Object:

- **Definition:** An object is a basic unit of Object-Oriented Programming and represents real-world entities. It is an *instance* of a class.
- **Nature:** It is a physical entity that consumes memory upon creation.
- **Characteristics:** Every object has three main characteristics:
 - **State:** Represents the data (value) of an object.
 - **Behavior:** Represents the functionality (methods) of an object.
 - **Identity:** An internally generated unique ID used by the JVM to identify the object.

Java Programming

Question 6: Explain access control and modifiers.

Definition: Access modifiers in Java help restrict the scope or visibility of a class, constructor, variable, method, or data member. They enforce the concept of Encapsulation by controlling who can access certain components of the code.

Types of Access Modifiers (From most restrictive to least restrictive):

1. private:

- The access level is restricted strictly within the class itself.
- Cannot be accessed from outside the class, not even by child classes.

2. Default (No keyword used):

- If you do not explicitly state a modifier, it takes the default access.
- The access level is restricted to within the same package. It cannot be accessed from outside the package.

3. protected:

- The access level is within the same package, AND outside the package *only* through child classes (via inheritance).
- Useful for establishing strict parent-child class relationships.

4. public:

- The access level is everywhere.
- It can be accessed from within the class, outside the class, within the package, and from outside the package.

Java Programming

Question 7: What is a Constructor? Explain types of Constructor.

Definition: A constructor in Java is a special type of method that is used to initialize an object. It is called automatically when an instance of a class is created using the new keyword.

- **Rules:**

1. The constructor name must be exactly the same as the class name.
2. It must **not** have an explicit return type (not even void).
3. It cannot be abstract, static, final, or synchronized.

Types of Constructors:

1. **Default Constructor (No-Argument Constructor):**

- A constructor that has no parameters.
- If you don't write any constructor, the Java compiler provides a default one behind the scenes. It initializes variables to their default values (e.g., 0 for int, null for String).

2. **Parameterized Constructor:**

- A constructor that takes specific parameters to initialize the object with different values at the time of creation.

Java Programming

Question 8: Explain garbage collection with example.

1. What is Garbage Collection? In languages like C or C++, the programmer is manually responsible for both the creation and destruction of objects using keywords like `new` and `delete` (or `malloc` and `free`). If a programmer forgets to destroy useless objects, it leads to a **Memory Leak**, eventually crashing the system.

Java solves this by handling memory management automatically. **Garbage Collection** is the process by which Java programs perform automatic memory management. The JVM runs a low-priority background daemon thread (the Garbage Collector) that looks for objects in the **Heap memory** that are no longer being used by the program and deletes them to free up space.

2. How does the Garbage Collector work? (The Mark and Sweep Algorithm) While you don't need to write a thesis on this for the exam, mentioning the algorithm scores high marks. The standard GC process works in two basic steps:

- **Mark:** The GC scans all object references and "marks" the objects that are still reachable (alive) by the application.
- **Sweep:** The GC identifies the unmarked objects (unreachable/dead) and reclaims the memory they occupy.

3. When does an object become eligible for Garbage Collection? An object is eligible for GC when no active thread can access it. In other words, it has zero references pointing to it. There are four main ways this happens:

- **Nullifying a reference:** `java Student s1 = new Student(); s1 = null; // The Student object is now eligible for GC`
- **Reassigning a reference:**
`Student s1 = new Student();`
`Student s2 = new Student();`
`s1 = s2; // The first object s1 pointed to is now eligible for GC`
- **Anonymous objects:** Objects created without a reference variable are immediately eligible after their single use.
Ex. `new Student().displayInfo();`

Java Programming

UNIT - 3

Question 9: Explain Inheritance with its types.

Definition: Inheritance is a core pillar of OOP where one class (the child or derived class) acquires the properties (fields) and behaviors (methods) of another class (the parent or base class).

- **Purpose:** It provides code reusability and establishes an "IS-A" relationship.
- **Keyword:** The extends keyword is used to inherit a class.

Types of Inheritance in Java (at the class level):

1. **Single Inheritance:** One child class inherits from exactly one parent class.
2. **Multilevel Inheritance:** A child class acts as a parent for another child class (e.g., Class A -> Class B -> Class C).
3. **Hierarchical Inheritance:** Multiple child classes inherit from a single parent class. *(Note: Java does not support Multiple Inheritance with classes to avoid ambiguity, but achieves it via Interfaces).*

EX. CODE:

```
// Parent Class
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}

// Child Class inheriting from Animal
class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        // Child class accessing its own method
        myDog.bark();
        // Child class accessing the parent's method (Code Reusability)
        myDog.eat();
    }
}
```

Java Programming

Question 10: Explain the use of 'this' keyword with example.

Definition: In Java, this is a reference variable that refers to the **current object** of a class. It is primarily used within an instance method or a constructor.

Primary Uses of the this keyword:

1. **To resolve variable shadowing (Most common):** When a local variable/parameter has the same name as an instance variable, this is used to distinguish the instance variable.
2. **To invoke current class constructor:** this() can be used to call one constructor from another inside the same class.
3. **To return the current class instance:** return this;
4. **To pass as an argument in a method call:** method(this);

EX. CODE:

```
class Student {
    int rollNo; // Instance variable
    String name; // Instance variable

    // Parameterized constructor
    Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }

    void display() {
        System.out.println("Roll No: " + rollNo + ", Name: " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(1, "Rahul");
        s1.display();
    }
}
```

Java Programming

Question 11: Explain the concept of method overloading.

Definition: Method Overloading is a feature in Java that allows a class to have more than one method with the exact same name, provided their parameter lists are different. It is a way to implement **compile-time polymorphism** (or static binding).

Rules for Method Overloading: You can overload a method by changing its signature in two ways:

1. By changing the **number** of parameters.
2. By changing the **data types** of the parameters. *(Note: Changing only the return type does not overload a method and will cause a compile-time error).*

EX. CODE:

```
class MathOperations {

    // 1. Method with two integer parameters
    int add(int a, int b) {
        return a + b;
    }

    // 2. Overloaded by changing the NUMBER of parameters
    int add(int a, int b, int c) {
        return a + b + c;
    }

    // 3. Overloaded by changing the DATA TYPE of parameters
    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        MathOperations math = new MathOperations();

        System.out.println(math.add(10, 20));    // Calls method 1
        System.out.println(math.add(10, 20, 30)); // Calls method 2
        System.out.println(math.add(10.5, 20.5)); // Calls method 3
    }
}
```

Java Programming

Question 12: Explain the concept of method overriding.

Definition: Method Overriding occurs when a child class (subclass) provides a specific implementation for a method that is already defined in its parent class (superclass). It is used to achieve **run-time polymorphism** (or dynamic binding).

Rules for Method Overriding:

1. The method must have the exact same name as in the parent class.
2. The method must have the exact same parameters (signature) as in the parent class.
3. There must be an "IS-A" relationship (Inheritance using the extends keyword).
4. The access modifier cannot be more restrictive than the overridden method.

EX. CODE:

```
// Parent Class
class Bank {
    int getRateOfInterest() {
        return 0;
    }
}

// Child Class 1
class SBI extends Bank {
    // Overriding the parent method
    @Override
    int getRateOfInterest() {
        return 7;
    }
}

// Child Class 2
class HDFC extends Bank {
    // Overriding the parent method
    @Override
    int getRateOfInterest() {
        return 8;
    }
}

public class Main {
    public static void main(String[] args) {
        Bank b1 = new SBI();
        Bank b2 = new HDFC();

        System.out.println("SBI Rate: " + b1.getRateOfInterest() + "%");
        System.out.println("HDFC Rate: " + b2.getRateOfInterest() + "%");
    }
}
```

Java Programming

Question 13: Explain Packages in Java. Explain the process of creation of packages.

Definition: A Package in Java is a mechanism used to group related classes, interfaces, and sub-packages together. It acts like a folder or directory structure in a file system.

Advantages:

1. **Prevents Naming Conflicts:** Two classes can have the same name if they are in different packages (e.g., java.util.Date and java.sql.Date).
2. **Access Protection:** Packages work with access modifiers (like default and protected) to secure classes.
3. **Modularity:** Makes searching and locating classes easier.

Types of Packages:

1. **Built-in Packages:** Provided by Java (e.g., java.lang, java.util, java.io).
2. **User-defined Packages:** Created by the programmer.

Process of Creation of Packages:

1. **Declare the package:** Use the package keyword at the very top of your Java source file.
2. **Create the class:** Write your class code normally.
3. **Compile with directory flag:** Compile the file using the -d flag, which tells the compiler to automatically create the corresponding directory structure.

EX. CODE:

```
// 1. Package declaration
package mypack;

public class MyClass {
    public void display() {
        System.out.println("Welcome to my package!");
    }
}
```

Compilation Command:

"javac -d . MyClass.java" (The "." denotes the current directory. This command creates a folder named mypack and puts MyClass.class inside it).

Java Programming

UNIT - 4

Question 14: Explain byte and character stream class with suitable example

Introduction to Streams: In Java, a stream is a continuous flow of data. The java.io package provides stream classes to perform Input and Output operations. They are broadly classified into two categories:

1. Byte Stream Classes:

- **Purpose:** Used to perform input and output of 8-bit bytes. They are ideal for reading and writing raw binary data, such as images, audio files, or executable files.
- **Base Classes:** All byte stream classes descend from InputStream and OutputStream.
- **Common Implementations:** FileInputStream, FileOutputStream.

2. Character Stream Classes:

- **Purpose:** Used to perform input and output of 16-bit characters. They automatically handle the translation of Unicode characters, making them ideal for reading and writing text files.
- **Base Classes:** All character stream classes descend from Reader and Writer.
- **Common Implementations:** FileReader, FileWriter.

EX. CODE:

```
import java.io.*;

public class StreamDemo {
    public static void main(String[] args) {
        try {
            // Writing to a file using Character Stream
            FileWriter writer = new FileWriter("output.txt");
            writer.write("Hello, this is a character stream example!");
            writer.close();
            System.out.println("Data written successfully.");

            // Reading from a file using Character Stream
            FileReader reader = new FileReader("output.txt");
            int character;
            System.out.print("File content: ");
            while ((character = reader.read()) != -1) {
                System.out.print((char) character);
            }
            reader.close();

        } catch (IOException e) {
            System.out.println("An error occurred: " + e.getMessage());
        }
    }
}
```

Java Programming

Question 15: Explain wrapper classes in Java.

Definition: A Wrapper class in Java provides the mechanism to convert a primitive data type into an object, and an object into a primitive data type. For every one of the 8 primitive data types in Java, there is a corresponding Wrapper class in the java.lang package (e.g., int -> Integer, char -> Character, double -> Double).

Why are they needed?

1. **Collections Framework:** Data structures in Java (like ArrayList, HashMap) strictly work with Objects, not primitive data types.
2. **Utility Methods:** Wrapper classes provide helpful methods, such as converting a String to an integer using Integer.parseInt().

Core Concepts:

- **Autoboxing:** The automatic conversion of a primitive data type into its corresponding wrapper class object by the Java compiler.
- **Unboxing:** The automatic conversion of a wrapper class object back into its corresponding primitive type.

Ex. Code:

```
public class WrapperDemo {
    public static void main(String args[]) {
        // 1. Autoboxing: Primitive to Object
        int num = 20;
        Integer obj = Integer.valueOf(num); // Explicit conversion
        Integer autoObj = num;             // Autoboxing (Internal)

        System.out.println("Object value: " + autoObj);

        // 2. Unboxing: Object to Primitive
        Integer myObj = new Integer(50);
        int val = myObj.intValue();        // Explicit conversion
        int autoVal = myObj;               // Unboxing (Internal)

        System.out.println("Primitive value: " + autoVal);
    }
}
```

Java Programming

Question 16: Explain Exception Handling in Java. Give types of exceptions along with their causes.

Definition: An Exception is an unwanted or unexpected event that occurs during the execution of a program (at runtime) that disrupts the normal flow of instructions. Exception Handling is a mechanism to handle these runtime errors so that the normal application flow is maintained.

The 5 Core Keywords:

1. try: The block of code that might throw an exception.
2. catch: The block of code that handles the exception.
3. finally: A block of code that is *always* executed, regardless of whether an exception is handled or not (used for cleanup).
4. throw: Used to explicitly throw an exception.
5. throws: Used in the method signature to declare that a method might throw exceptions.

Types of Exceptions and Their Causes:

1. Checked Exceptions (Compile-time Exceptions):

- These are checked by the compiler at compile-time. The program will not compile if these are not handled.
- *Example:* IOException (caused by failed or interrupted I/O operations), SQLException (caused by database access errors).

2. Unchecked Exceptions (Runtime Exceptions):

- These are not checked at compile-time but occur at runtime. They usually represent programming logic errors.
- *Examples:* * ArithmeticException: Caused by mathematical errors, like dividing a number by zero (int a = 50/0;).
 - NullPointerException: Caused by invoking a method or property on a null object reference.
 - ArrayIndexOutOfBoundsException: Caused by accessing an array with an invalid/non-existent index.

3. Errors:

- Irrecoverable conditions that are beyond the control of the programmer (e.g., OutOfMemoryError, StackOverflowError).

Question 17: Compare throw and throws in Java.

Java Programming

Feature	throw keyword	throws keyword
Definition	Used to explicitly throw a single exception from within a method.	Used in the method signature to declare that the method might throw exceptions.
Location	Used <i>inside</i> the method body.	Used <i>with the method signature</i> .
Followed by	Followed by an instance (object) of an Exception class.	Followed by the name of the Exception class(es).
Multiple Exceptions	You cannot throw multiple exceptions at the same time.	You can declare multiple exceptions separated by commas.
Propagation	It does not propagate checked exceptions to the calling method automatically.	It propagates checked exceptions up the call stack to the method that invoked it.

```
class Demo {  
    // 'throws' used in method signature  
    static void checkAge(int age) throws ArithmeticException {  
        if (age < 18) {  
            // 'throw' used inside the method body  
            throw new ArithmeticException("Access denied - You must be at least 18.");  
        } else {  
            System.out.println("Access granted.");  
        }  
    }  
  
    public static void main(String[] args) {  
        try {  
            checkAge(15);  
        } catch (ArithmeticException e) {  
            System.out.println("Exception caught: " + e.getMessage());  
        }  
    }  
}
```

Java Programming

UNIT - 5

Question 18: What are Java Threads? Explain life cycle of Thread.

Definition: A Thread in Java is a lightweight sub-process, the smallest independent unit of processing. Multithreading is the process of executing multiple threads simultaneously to maximize CPU utilization. Java provides built-in support for multithreading through the `java.lang.Thread` class.

Life Cycle of a Thread: A thread in Java goes through several states from its creation to its termination, controlled by the JVM.

1. New (Born State):

- A thread is in this state when an instance of the Thread class is created, but the `start()` method has not yet been invoked on it.

2. Runnable (Ready State):

- When the `start()` method is called, the thread becomes Runnable. It is placed in the runnable pool and is waiting for the Thread Scheduler to allocate CPU time to it.

3. Running:

- The thread moves into the Running state when the Thread Scheduler selects it from the runnable pool and it begins executing its `run()` method.

4. Blocked / Waiting (Non-Runnable State):

- A thread transitions here when it is temporarily inactive. It might be waiting for I/O resources, waiting for a locked monitor (synchronization), or put to sleep using `Thread.sleep()`. Once the condition is met, it returns to the Runnable state.

5. Terminated (Dead State):

- A thread enters this state when its `run()` method completes its execution normally, or if an unhandled exception causes it to terminate prematurely. Once dead, a thread cannot be restarted.

Java Programming

Question 19: What is multithreading? In how many ways Java implements multithreading? Explain with example.

Definition: Multithreading is a Java feature that allows concurrent execution of two or more parts of a program for maximum utilization of the CPU. Each part of such a program is called a thread.

Ways to Implement Multithreading in Java: There are two primary ways to create a thread in Java:

1. **By extending the Thread class.**
2. **By implementing the Runnable interface** (This is generally preferred because Java does not support multiple inheritance, so implementing an interface leaves your class free to extend another class if needed).

Ex. Code:

```
// 1. Extending the Thread class
class MyThread extends Thread {
    // 2. Overriding the run() method
    public void run() {
        for(int i = 1; i <= 3; i++) {
            System.out.println("Child Thread executing: " + i);
            try {
                Thread.sleep(500); // Pauses thread for 500ms
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class MultithreadDemo {
    public static void main(String args[]) {
        MyThread t1 = new MyThread();

        // 3. Starting the thread
        t1.start(); // This internally calls the run() method

        for(int i = 1; i <= 3; i++) {
            System.out.println("Main Thread executing: " + i);
        }
    }
}
```

Java Programming

Question 20: How can Inter Thread Communication be done in Java?

Definition: Inter-thread communication is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) the same critical section to be executed. It allows synchronized threads to communicate with each other, preventing "polling" (where a thread repeatedly checks a condition, wasting CPU cycles).

How it is done: Java achieves inter-thread communication using three final methods defined in the Object class (not the Thread class):

1. **wait():**

- Causes the current thread to release its lock and wait until another thread invokes the notify() or notifyAll() method for this object.

2. **notify():**

- Wakes up a single thread that is waiting on this object's monitor. If multiple threads are waiting, one is chosen at random.

3. **notifyAll():**

- Wakes up all threads that are waiting on this object's monitor.

Java Programming

Question 21: Explain suspending and resuming thread in Java with example.

Concepts:

- **Suspending a thread:** The `suspend()` method of the `Thread` class is used to temporarily stop the execution of a thread. A suspended thread puts the thread in a waiting state and does not consume CPU cycles, but it *keeps hold of its monitors (locks)*.
- **Resuming a thread:** The `resume()` method is used to revive a suspended thread. It moves the thread from the waiting state back to the runnable state so it can continue execution.

```
class MyThread extends Thread {
    public void run() {
        try {
            for (int i = 1; i <= 5; i++) {
                System.out.println("Thread running: " + i);
                Thread.sleep(500);
            }
        } catch (InterruptedException e) {
            System.out.println("Exception: " + e);
        }
    }
}

public class ThreadControlDemo {
    Run | Debug
    public static void main(String args[]) {
        MyThread t1 = new MyThread();
        t1.start();

        try {
            Thread.sleep(1000);

            System.out.println(x: "Suspending thread...");
            t1.suspend(); // Temporarily stops t1

            Thread.sleep(2000); // Main thread waits 2 seconds

            System.out.println(x: "Resuming thread...");
            t1.resume(); // Restarts t1 from where it left off
        } catch (InterruptedException e) {
            System.out.println("Exception: " + e);
        }
    }
}
```